Proposal: reduce per-group allocation in `_build_protein_rollup`

branch `perf/alloc-diagnostics`

2026-06-24

1 Summary

`_build_protein_rollup` (in `src/Routines/SearchDIA/SearchMethods/ProteinScoringSearch/protein_infer`
is called **once per protein group** inside two `groupby` loops. Each call allocates **14 `Dict`s plus several row vectors**; over all groups × files this is ∼0.6 GB on the YeastStandard3M benchmark (the largest single site in the Protein Scoring phase, which totals 1.98 GB).

The fix is to (a) replace the 14 `Dict`s with **3 dedup `Dict`s + reusable parallel arrays**, and (b) hoist that scratch out of the per-group function so it is **allocated once and reused** across groups. Net per-group allocation drops from "14 Dicts + vectors" to "the returned output rows only".

Caveat (please note): the current function emits its rows in `Dict`-hash iteration order, and `pg_score` is an *order-sensitive* `Float32` sum (§4). The new design iterates in deterministic first-seen order instead, so this is a **deliberate, one-time protein-scoring rebaseline** (a few protein groups may shift, same class as the approved iRT-refinement rebaseline). It also *removes* the latent order-fragility permanently. Verification plan in §8.

2 Where it is called (per protein group)

Loop 1 — `protein_inference_pipeline.jl:664--666`:

```
664 for gdf in groupby(df, [:inferred_protein_group, :target, :entrap_id])
665     quant_mask = _protein_rollup_quant_mask(gdf; q_value_threshold = q_value_threshold)
666     rollup = _build_protein_rollup(gdf, quant_mask, prob_col)
```

Loop 2 — `:1124--1126` (inside `combine(grouped) do gdf ... end`):

```
1124 protein_groups = combine(grouped) do gdf
1125     quant_mask = _protein_rollup_quant_mask(gdf; q_value_threshold = q_value_threshold)
1126     rollup = _build_protein_rollup(gdf, quant_mask, prob_col)
```

Both call once per `(inferred_protein_group, target, entrap_id)` group, for each run/file. Both loops are sequential (a plain `for`, and DataFrames `combine`, which is single-threaded), so a single reusable scratch instance per loop is safe.

3 Current implementation (the 14 Dicts)

Per-precursor level — 7 Dicts (`:427--459`)

```
prob_by_precursor            = Dict{UInt32, Float32}()
best_peak_area_by_precursor  = Dict{UInt32, Float32}()
base_pep_id_by_precursor     = Dict{UInt32, UInt32}()
sequence_by_precursor        = Dict{UInt32, String}()
charge_by_precursor          = Dict{UInt32, UInt8}()
structural_mods_by_precursor = Dict{UInt32, String}()
isotopic_mods_by_precursor   = Dict{UInt32, String}()

@inbounds for i in eachindex(quant_mask)
    quant_mask[i] || continue
    precursor_idx = UInt32(gdf.precursor_idx[i])
    prob_val      = _sanitize_rollup_probability(gdf[!, prob_col][i])
    peak_area_val = Float32(gdf.peak_area[i])
    if haskey(prob_by_precursor, precursor_idx)
        prob_by_precursor[precursor_idx] = max(prob_by_precursor[precursor_idx], prob_val)
        if peak_area_val > best_peak_area_by_precursor[precursor_idx]
            best_peak_area_by_precursor[precursor_idx] = peak_area_val
        end
    else
        prob_by_precursor[precursor_idx]           = prob_val
        best_peak_area_by_precursor[precursor_idx] = peak_area_val
        base_pep_id_by_precursor[precursor_idx]    = UInt32(gdf.base_pep_id[i])
        sequence_by_precursor[precursor_idx]       = String(gdf.sequence[i])
        charge_by_precursor[precursor_idx]         = hasproperty(gdf, :charge) ? UInt8(gdf.charge[i
    ]) : UInt8(0)
        structural_mods_by_precursor[precursor_idx] = ...   # String or ""
        isotopic_mods_by_precursor[precursor_idx]   = ...   # String or ""
    end
end
```

The output rows are then built by iterating `keys(prob_by_precursor)` (**:463**) — i.e. in `Dict`-hash order.

Per-modified-peptide level — 4 Dicts (:493--496)

```
modified_log_none_sum   = Dict{Tuple{UInt32, String, String}, Float64}()
modified_best_peak_area = Dict{Tuple{UInt32, String, String}, Float32}()
modified_sequence       = Dict{Tuple{UInt32, String, String}, String}()
modified_precursor_count= Dict{Tuple{UInt32, String, String}, Int32}()
```

Per-peptide level — 3 Dicts (:529--531)

```
peptide_log_none_sum   = Dict{String, Float64}()
peptide_best_peak_area = Dict{String, Float32}()
peptide_modified_count = Dict{String, Int32}()
```

Total: **7 + 4 + 3 = 14 Dicts allocated per call**. Measured contribution (`Profile.Allocs`, sampled): ∼0.6 GB across lines 438–540.

4 The order fragility

`pg_score` is a `Float32` sum over `peptide_rows` (**:556**):

```
pg_score = isempty(peptide_rows) ? 0.0f0 : Float32(sum(row.score for row in peptide_rows))
```

`peptide_rows` is built by iterating `keys(peptide_log_none_sum)` (**:545**), i.e. `Dict`-hash order. Floating-point summation is not associative, so the iteration order changes `pg_score` in the last bits. `pg_score` feeds protein-group FDR, so a last-bit change can flip a borderline protein near the q-value threshold. *This is why naive scratch reuse (`empty!` changes a Dict's capacity, hence its iteration order) is not byte-for-byte result-neutral* — the same mechanism as the iRT `sizehint!` issue. (`peptide_list` at :558 is `sort!`'d so it is already order-independent; `top_pep_peak_area` is a `maximum`, also fine.)

5 Why not merge the Dicts into one struct-valued Dict?

A natural idea is `Dict{UInt32, PrecAccum}`. It does not help: the per-precursor record holds `String` fields (`sequence`, `structural_mods`, `isotopic_mods`), so `PrecAccum` is not `isbits` and the `Dict` would box one struct *per precursor* — worse than 7 scalar/String Dicts.

6 Proposed change

Use **one dedup `Dict` per level** (precursor_idx / mod-key / sequence → a row index) plus **parallel arrays** for the fields, pushed in **first-seen order**. Iterate the arrays by position (deterministic, capacity-independent). Hold all of this in a `ProteinRollupScratch` that the caller allocates once and passes in; `_reset!` it (`empty!` the 3 Dicts + the arrays, keeping their capacity) at the start of each call.

Scratch type (allocated once per loop, reused)

```
mutable struct ProteinRollupScratch
    # precursor level (dedup by precursor_idx; parallel arrays, first-seen order)
    prec_pos    :: Dict{UInt32, Int}
    prec_id     :: Vector{UInt32}
    prec_prob   :: Vector{Float32}
    prec_peak   :: Vector{Float32}
    prec_basepep:: Vector{UInt32}
    prec_seq    :: Vector{String}
    prec_charge :: Vector{UInt8}
    prec_smods  :: Vector{String}
    prec_imods  :: Vector{String}
    # modified-peptide level (dedup by (base_pep_id, structural_mods, isotopic_mods))
    mod_pos     :: Dict{Tuple{UInt32,String,String}, Int}
    mod_key     :: Vector{Tuple{UInt32,String,String}}
    mod_logsum  :: Vector{Float64}
    mod_peak    :: Vector{Float32}
    mod_seq     :: Vector{String}
    mod_count   :: Vector{Int32}
    # peptide level (dedup by sequence)
    pep_pos     :: Dict{String, Int}
    pep_seq     :: Vector{String}
    pep_logsum  :: Vector{Float64}
    pep_peak    :: Vector{Float32}
    pep_count   :: Vector{Int32}
end
```

Per-precursor accumulation (replaces :427–459)

```
_reset!(s)   # empty! the 3 Dicts and all arrays (capacity retained for reuse)
@inbounds for i in eachindex(quant_mask)
    quant_mask[i] || continue
    pid       = UInt32(gdf.precursor_idx[i])
    prob_val  = _sanitize_rollup_probability(gdf[!, prob_col][i])
    peak_val  = Float32(gdf.peak_area[i])
    pos = get(s.prec_pos, pid, 0)
    if pos == 0                             # first time we see this precursor
        push!(s.prec_id, pid)
        push!(s.prec_prob, prob_val)
        push!(s.prec_peak, peak_val)
        push!(s.prec_basepep, UInt32(gdf.base_pep_id[i]))
        push!(s.prec_seq, String(gdf.sequence[i]))
        push!(s.prec_charge, hasproperty(gdf, :charge) ? UInt8(gdf.charge[i]) : UInt8(0))
        push!(s.prec_smods, _mods_or_empty(gdf, :structural_mods, i))
        push!(s.prec_imods, _mods_or_empty(gdf, :isotopic_mods, i))
        s.prec_pos[pid] = length(s.prec_id) # row index in the parallel arrays
    else                                    # seen before: update aggregates only
        s.prec_prob[pos] = max(s.prec_prob[pos], prob_val)
        if peak_val > s.prec_peak[pos]; s.prec_peak[pos] = peak_val; end
    end
end
# build precursor_rows by iterating 1:length(s.prec_id)  (first-seen order)
```

The modified-peptide and peptide levels follow the identical pattern (one dedup `Dict` + parallel arrays each). The returned `precursor_rows / modified_peptide_rows / peptide_rows` are still built (they are the output the callers consume) — only the *intermediate* 14 Dicts become 3 reused dedup Dicts + reused arrays.

Caller change (allocate scratch once)

```
scratch = ProteinRollupScratch()                    # once, before the loop
for gdf in groupby(df, [...])
    quant_mask = _protein_rollup_quant_mask(gdf; ...)
    rollup = _build_protein_rollup(gdf, quant_mask, prob_col, scratch)   # 4-arg
end
```

A 3-arg method `_build_protein_rollup(gdf, quant_mask, prob_col)` is kept (allocates a fresh scratch then calls the 4-arg form) so the existing signature and any tests keep working.

7 Allocation impact (expected)

Per group: **14 Dict allocations → 0** (3 dedup Dicts + 14 arrays all reused from the scratch). The only remaining per-group allocations are the returned output row vectors (necessary). Expected saving on YeastStandard3M: ∼0.3–0.5 GB of the 0.6 GB (to be confirmed by the `@alloc_bucket/Profile.Allocs` re-run).

8 Correctness, rebaseline, and verification

- **Aggregates unchanged:** `max/sum/count` per precursor/peptide are identical regardless of iteration order; only the *order* in which rows are emitted changes (Dict-hash → first-seen).

- **Deliberate rebaseline:** because `pg_score` sums in row order, the new first-seen order changes `pg_score` in the last bits, so a few borderline protein groups may cross the FDR threshold. This is a one-time rebaseline (and removes the latent order-fragility for good — the result becomes a deterministic function of the input, not of Dict capacity).
- **Verification:** fresh-process determinism (the new protein-group / precursor counts must be *reproducible* across runs); confirm the change vs the current baseline is small (1 %, like the iRT rebaseline) — a large change would signal a bug. Re-run `@alloc_bucket` / `Profile.Allocs` to confirm the allocation drop and no Protein Scoring time regression. Existing ProteinScoringSearch unit tests must pass.

9 Optional follow-on (not in this change)

The returned `*_rows` `Vector{NamedTuple}` are still allocated per group. If the callers consumed the scratch parallel arrays directly, those could be avoided too — but that touches the caller contract and is left out to keep this change focused.